Is grep all you need for RAG?

APRIL 6TH, 2026

Recently Mintlify shared how they replaced RAG with grep over a virtual file system (https://www.mintlify.com/blog/how-we-built-a-virtual-filesystem-for-our-assistant):

> *RAG is great, until it isn't.*
>
> *Agents are **converging on filesystems as their primary interface** (https://arxiv.org/abs/2601.11672) because `grep`, `cat`, `ls`, and `find` are all an agent needs. If each doc page is a file and each section is a directory,*

RAG is dead, again. 😉

In reality, agents can get by with "dumb retrieval tools" (grep, naive keyword search (https://softwaredoug.com/blog/2025/10/06/how-much-does-reasoning-improve-search-quality)).

Why? ***Constraints***. Constraints force the agent to budget creativity: avoiding pointless search strategies; saving reasoning for where it matters.

They show up *everywhere*.

We have structured outputs + tool calls (https://developers.openai.com/api/docs/guides/structured-outputs). For example, we might force an agent to choose a legal category filter in naive keyword search, as in the tool below:

```
def naive_keyword_search(keywords: str,
                         categories: Literal['furniture', 'electronics',
                                             'fashion', ...]):
    """Search simple keyword search, filtered to the required category."""
```

Then there's a more implicit constraint: the training data itself. (https://mattstockton.com/2025/03/11/practical-building-with-llms.html#the-little-red-riding-hood-principle) Since frontier labs care about navigating code, we can confidently say `grep` exists on that happy path.

But lets really dive into the overriding, big kahuna constraint, hooks**.**

The big constrainer: hooks

Even with these modest attempts to keep agents on the rails, the inner agentic loop is a beast to tame. That's why the big constraint comes from *hooks* **-** programmatic responses to the search agent on behalf of the user.

How does this work? An agent works to gather results (via grep or some other tool). It iterates until happy. It responds with a list of search results. **Done!** (right?)

Not so fast, Mr. Agent. Time to check your search results.

Is each result recent? popular? an authoritative source? (add whatever matters to your domain)

We check these *programmatically.* Our hook sees the agent it's failed to meet our bar of quality.

Just as a user would manually express disappointment with result quality, the hook now responds automatically. Saying back (on behalf of the agent) "I'm disappointed with you Mr. Agent, these results don't meet this bar / that bar in quality, try harder."

This could be as sophisticated as we want - a reranker or LLM-as-a-judge. However you do it, enforcing a high bar for the ideal results lets you push the agent to try its darndnest to satisfy your ideal, best-case scenario - grep or otherwise.

And if it can't? Well you can always fall back to results from those earlier rounds.

Search harness design

The combination of all your constraints (hooks, tool signatures, etc) comes together in a ***harness***.

I think of the harness as

- ***Tools*** - the actual retrieval functionality you use, grep or otherwise.
- ***State*** - used by hooks and tools to enforce any needed constraints. Perhaps letting a tool give an error if an agent repeats similar searches to encourage exploration
- ***Hooks*** - validation conditions for accepting the agent's work

We end up with an inner loop and an outer loop. The inner loop calls the LLM, iterating until it's exhausted the LLM's own request for function calls. It's us working until the LLM is satisfied.

The outer loop validates search results to see if they meet our high bar. If not, we keep telling the agent to "try harder" with useful guidance. It's about the LLM working until WE are satisfied.

The inner loop looks (very roughly) like the following snippet:

```
def agent_loop(inputs):
       tool_calls_found = True
       while tool_calls_found:
             # The initial call (diagram one)
             # (But also subsequent calls with tool responses appended to `input`
             resp = self.openai.responses.parse(
                 model=self.model,
                 input=inputs,   # All the user, tool, agent interactions so far
                 tools=tools     # grep maybe?
              )
              inputs += resp.output

                  for item in resp.output:
                      # call tools, package up response
                      ...
              # return the state of the system
              return inputs
```

Then we need an outer loop, driving this one, checking it, ensuring it's meeting our needs

```
system_prompt = "You are a helpful search agent that ..."
def harness(user_prompt):
    """Drive the agent loop until we're happy."""
            inputs = [
                {"role": "system", "content": system_prompt},
                {"role": "system", "content": user_prompt},
            ]
            valid = False
            while not valid:
                inputs = agent_loop(inputs)
                search_results = inputs[-1]
                # ****
                # Check if we like the results
                for result in search_results:
                    # Here we validate. with some rules, but
                    # You could do anything, make it query dependent
                    # Or even have an LLM generate what's acceptable
                    if result.review < 4.0:
                       inputs.append({"role": "system",
                                      "content": "Result {result} is not high er
               valid = False
               continue
        ...
```

In a way we've deconstructed the search stack.

Traditionally all these bits would be behind the search functionality. These "dumb retrieval tools" would be inputs to a reranking pipeline from backend systems (ie your Elasticsearch). And the rules / rerankers job would shuffle the initial ranking to put relevant results above less relevant.

Now, for the agents benefit, we've taken it all apart. We've just given it raw access (ie `grep`) and then sprinkled acceptance criteria into the outer loop.

Why retrieval quality still matters

Nothing here eliminates the need for high quality retrieval.

Yes we've deconstructed the search stack. Moving logic into the harness we might otherwise keep behind the search tool itself. That's useful guidance to an agent trying to answer user questions.

But in the end, feedback that's not actionable doesn't help an agent. Berating an agent using search tool that can't prioritize what's relevant won't help anything.

You CAN with enough effort design any system. Modeling relevance factors like recency, popularity in a greppable markdown file might be fun. Or even appropriate at a small scale.

But modern retrieval balances ranking half a dozen of these factors. It works hard to rank them against each other - alongside text relevance factors (embeddings + lexical search). Often text relevance matters little - other times quite a lot.

In short, it gets complex fast.

Agentic search - like coding - means constraining our dumb LLM intern.

Yes building with naive retrieval points the way to robust harnesses. But eventually you'll regret the token cost of many tool calls to make it all work.

It turns out agents DO have one appropriate tool in their training data - it's called `search` ;)

-Doug

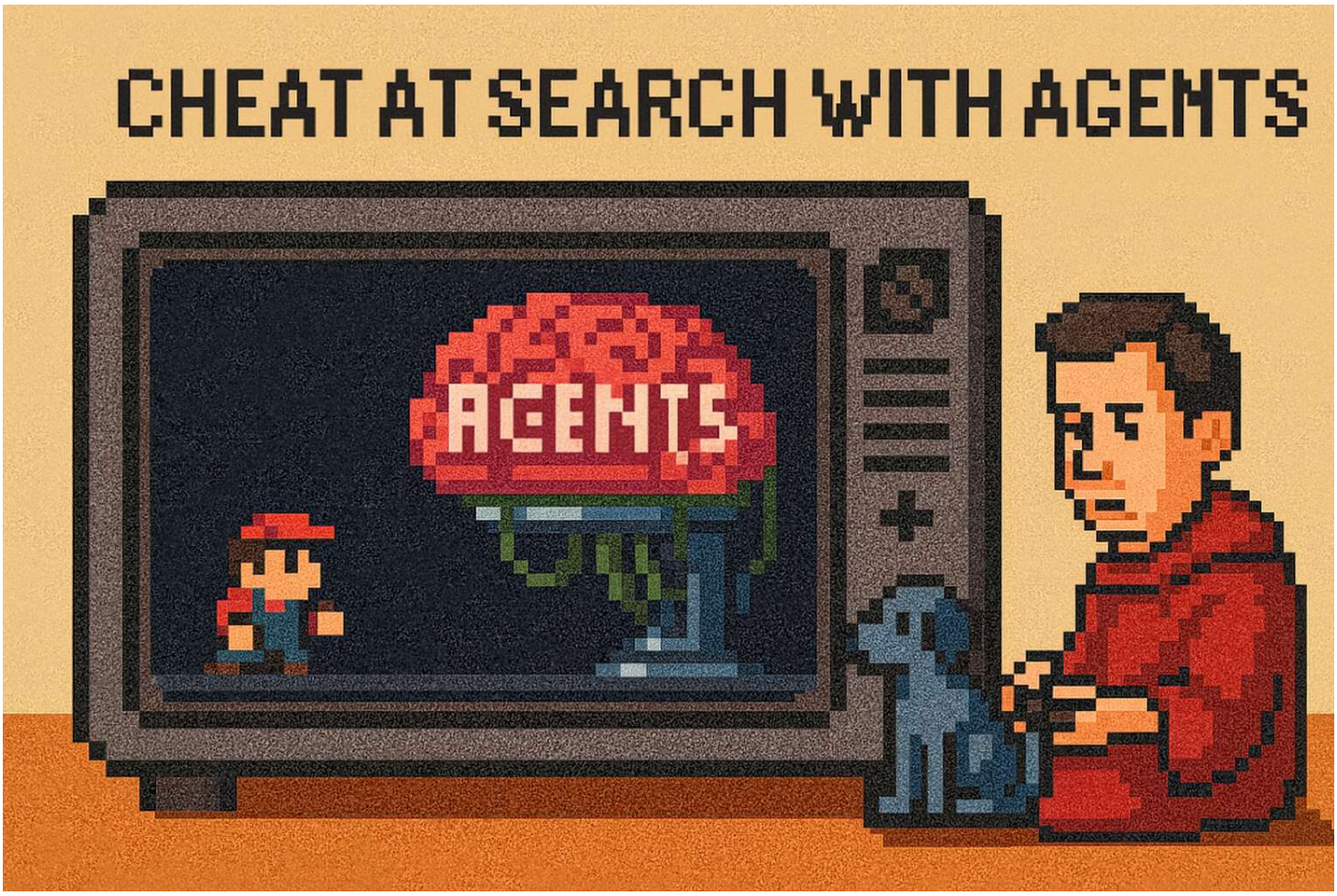
Would you like to know more? Last day before prices increase on Cheat at Search with Agents (https://maven.com/softwaredoug/cheat-at-search), my agentic search course.

This is part of Doug's Daily Search tips - subscribe here (http://softwaredoug.kit.com)

Enjoy softwaredoug in training course form!

Starting May 18!

Signup here - http://maven.com/softwaredoug/cheat-at-search (http://maven.com/softwaredoug/cheat-at-search)



(https://maven.com/softwaredoug/cheat-at-search)

I hope you join me at Cheat at Search with Agents (https://maven.com/softwaredoug/cheat-at-search) to learn use agents in search. build better RAG and use LLMs in query understanding.

Doug's search blog and newsletter

Doug Turnbull

More from Doug (/)
Twitter (https://twitter.com/softwaredoug/) | LinkedIn (https://www.linkedin.com/in/softwaredoug/) | Newsletter (https://softwaredoug.kit.com) | Bsky (https://bsky.app/profile/softwaredoug.bsky.social)

Learn more (http://softwaredoug.kit.com)

Doug's search blog and newsletter

Learn more (http://softwaredoug.kit.com)